Study 2 follow-up: active-node and decision-utility mutation

This is a post-study repair and ablation motivated by the earlier inspected Study 2 result. It was not planned before that result. Study 1's robust instruction fitness and Study 2's original complementary-error guidance both retain their negative findings. No publication-level novelty is claimed for adaptive stopping, informed mutation, utility search or evolutionary computation.

Research question and evaluation

Does restricting mutation to observed search states and scoring complete-policy utility improve beyond ordinary mutation, active-node random mutation and exhaustive ordered-triple controls? The model/instruction, five formats, canonical identities, singleton decoding, tree limits, crossover and multi-objective survival remain fixed. Search and validation caches are reused. The newly reserved final pool is 1000 eligible previously unqueried questions from the official OpenBookQA TRAINING split; it is not the official test split. The previous inspected 486-question official-test pool was excluded and is exploratory for this redesign.

The primary operating point is a three-call validation cap. All figures report achieved final cost, which can differ from the cap. Candidate selection maximizes validation accuracy under the cap, then calls, size and hash; utility steers mutation, not this validation rule. Evolutionary and hill-climbing entries average five fitted seeds. Only frozen policies were evaluated on the final pool.

Primary results: all three required outcomes

G2 minus G1 accuracy: +0.000000, paired conditional 95% interval [+0.000000, +0.000000]; achieved calls 2.186 versus 2.180.
G2 minus G1 calls: +0.006400, paired conditional 95% interval [-0.002405, +0.015800]; achieved calls 2.186 versus 2.180.
G2 minus G1 utility: -0.000064, paired conditional 95% interval [-0.000158, +0.000024]; achieved calls 2.186 versus 2.180.

The observed mean utility difference does not favor G2 under the predeclared 0.01 preference. This negative/null finding is retained without further redesign.

Method	Accuracy	Mean calls	Utility	Requested tokens
G0	83.30%	2.348	0.8095	237.8
G1	83.30%	2.180	0.8112	220.6
G2	83.30%	2.186	0.8111	221.4
G3	83.30%	2.188	0.8111	221.6
fixed_agreement	79.10%	2.093	0.7701	215.2
fixed_full	80.60%	5.000	0.7560	513.1
fixed_subset	83.30%	3.000	0.8030	303.7
fixed_three	83.30%	3.000	0.8030	303.7
hill	83.30%	2.212	0.8109	223.9
legacy_greedy	83.40%	2.261	0.8114	228.9
ordered_triple	83.30%	2.180	0.8112	220.6
single	80.10%	1.000	0.7910	101.2

Mechanism diagnostics and budget accounting

mechanism_diagnostics.csv reports every condition/seed's zero-visit frequency, explicit random/global-fallback rates and reasons, prediction/call changes, exploration frequency, trial/selected utility changes and probes per accepted offspring. Raw node visits and every scored trial's utility changes are in mutation_diagnostics.jsonl and the durable search databases. Accepted offspring means a proposal enters the offspring batch, before survival; hill climbing records actual accepted moves. Zero visits means unobserved on the finite search pool, not logically unreachable. Unobserved branches are not removed on this basis.

Every initial, base and scored trial policy replays the complete search pool and consumes one evaluation equivalent, including duplicates and discarded probes. Common budget and actual raw/question replay counts, candidate counts, full/partial generations, replay timing, search wall/CPU time and interrupted-attempt statuses are in search_costs.csv. G2 gets fewer offspring because its probes are charged. Validation and post-search diagnostic replays are separate from search and never feed back into proposals. Validation performed 42378 policy evaluations / 20722842 question-policy replays. Selected-policy diagnostic search replays: 49152. Selection timing is retained in selection_costs.json. The existing greedy baseline is retained from the prior study; matched-budget utility hill climbing provides the new local-search control.

Uncertainty, limits and interpretation

Whole-question paired bootstrap uses the same resampled question indices for methods, averaging fitted-seed outcomes inside each question. Accuracy, calls and U are all reported for G2 versus G1; no favorable metric is selected after seeing results. Intervals condition on fitted policies and exclude retraining uncertainty. Search-seed mean/SD/range and per-seed paired differences are separate. No seed-question independence assumption or interval-overlap significance test is used. Test frontiers are descriptive and never reselect

deployment policies.

Only one fixed model, a filtered dataset, five presentations, finite trees, a fixed budget and five seeds are studied. Exact duplicate checks do not remove every paraphrase or pretraining overlap. Validation selection over many candidates can overfit. G2's bounded utility trial pool differs from the legacy proposal distribution as documented in the protocol; its G1 comparison therefore tests the complete guided proposal mechanism. The hill control uses the same trial pool/scorer to examine local improvement without population crossover. A newly held-out official-training subset is not a standard official-test benchmark score. Requested-call/token reductions from replay are not established production-latency savings.

Actual costs

Cache costs by split: {"search": {"completed_unique_calls": 2560, "executed_here_calls": 0, "input_tokens": 257151, "output_tokens": 5121, "inference_seconds": 371.0443683862686, "invalid_outputs": 1, "reused_previous_calls": 2560, "note": "Generation tokens/time are historical source costs, not newly executed follow-up inference."}, "validation": {"completed_unique_calls": 2445, "executed_here_calls": 0, "input_tokens": 250281, "output_tokens": 4890, "inference_seconds": 353.9433799088001, "invalid_outputs": 0, "reused_previous_calls": 2445, "note": "Generation tokens/time are historical source costs, not newly executed follow-up inference."}, "test": {"completed_unique_calls": 5000, "executed_here_calls": 5000, "input_tokens": 503131, "output_tokens": 10007, "inference_seconds": 734.9938200861216, "invalid_outputs": 4, "reused_previous_calls": 0}}. Imported search/validation costs are historical and separated from newly executed final inference. CPU-search accounting is in search_costs.csv; cache-generation attempts, raw responses and output tokens are retained in responses.sqlite/costs.json. Small development verification and CPU benchmarks are separately exported in preparation_costs.json. Policies/configuration/source/analysis hashes were saved before generating final responses. Only COMPLETE.json marks successful completion.

Report integration and submission

Use this section as a labeled follow-up within Study 2. Preserve the original null/negative findings and disclose that they motivated the repair. The complete original reports remain in the parent project and old_idea. This PDF includes the detailed follow-up protocol and existing thematic review of 22 recent primary-source works. The course brief requires a self-contained final PDF, code/README, experimental outputs, baselines, multiple runs/statistics and at least twenty 2024–2026 papers. The final combined course report still needs human editorial integration, figure/readability checks and submission packaging; automatic artifact creation is not final submission.

Follow-up protocol, frozen before new final inference

This is one bounded post-study repair within Study 2, motivated by previously inspected results. It is not a new direction or a retrospectively planned part of the original study. Study 1's robust-fitness prompt result and Study 2's original guidance result are preserved. Preparation began 29 October 2026 at 20:26 Asia/Shanghai, with a four-hour implementation/verification limit. The user reserves 30 October for writing. The supplied course brief's submission deadline is 15 November 2026.

Data and fixed model

One fixed Qwen/Qwen2.5-3B-Instruct model/tokenizer revision aa8e72537993ba99e69dfaafa59ed015b17504d1; fixed manual instruction and exact answer contract inherited verbatim. Five formats: original, inline, bullets, rotate1, rotate2. Canonical identities are the original option indices. Strict uppercase A–D parsing, invalid abstention, lowest canonical-index vote ties, explicit all-invalid failure, no input truncation, greedy singleton BF16 generation, seed zero, eight new tokens maximum, deterministic Torch settings, SDPA and 1,536 input-token limit remain unchanged. Policies observe only queried predictions, agreement, invalids, already requested formats and remaining calls. Ground truth never enters deployment state or action selection.

Search (512) and selection-validation (489) are exact reused Study 2 source questions/caches, whose content, model, tokenizer, instruction, rendering, permutation, decoding and cache keys are verified. The inspected 486-question official-test pool is excluded from the new final assessment. All historical split files and ten prediction-cache databases are inventoried in data/history_inventory.json, including old real-model smokes. Previous source IDs, normalized question stems, unordered question/option content, duplicate options, likely position references and missing media are screened. The inventory found 4,146 eligible unused official-training questions. Exactly 1,000 are held out by reservation seed 20261029; full identifiers, hashes, exclusion reasons and source parquet checksum are saved. This is a newly held-out subset of official OpenBookQA TRAIN, not official test. Exact screening cannot guarantee absence of paraphrases or pretraining exposure. Raw-data screening is not model evaluation. New final previews and predictions are unavailable before policy freeze.

Representation and common evolution

Inherited binary query trees have depth <=5, <=15 query nodes and <=5 calls, with no repeated format on any path. Null children are stop actions. The same predicates, canonical voting, constraint repair,

initialization, every-fourth-offspring subtree crossover, rank/hash tournament and nondominated accuracy/call survival with crowding are retained. This is NSGA-II-style selection, not a claim of a faithful reference implementation. Node count is a bound, not a third objective.

Five matched seeds: 11,29,47,71,101; population 32. Shared initial populations contain five singles, the original agreement policy and full-five voting, plus the same seeded random trees. All four evolutionary conditions use these identical populations per seed. Utility hill climbing receives the same initialization but follows a single incumbent, without crossover or population survival. The finite evaluation budget, rather than a fixed generation count, ends search; methods can therefore have different offspring/generation counts.

Variants

G0 is the original ordinary mutation distribution over all tree sites. G3 is the original complementary-error mutation distribution and its documented global search-failure fallback. Their formats, operation choices and random-number behavior are regression-checked against the inherited operators. G3's failure statistics reuse the already charged base-policy predictions/visits, avoiding an unnecessary identical replay while retaining exactly the original rescue weights and choices.

G1 changes the location sampling of ordinary random mutation: choose uniformly among sites reached by at least one search question at the current post-crossover policy. Both query nodes and extendible stop sites are eligible. Terminal sites at depth five are omitted because no legal distinct query extension exists. G2 uses exactly the same active-site sampling rule. Zero visits means unobserved in finite search data, not logically unreachable. No branch is deleted on the basis of a zero visit count; only inherited depth/size/no-repetition repair may alter it when an edit creates a constraint conflict.

G2 constructs at most eight distinct legal complete trial policies at the chosen active site. First include every change to another path-unused query format, or each one-query extension at a stop site; include stopping at a non-root query node where legal. Root stopping without any observation is illegal. Then include one seeded random-subtree proposal and shuffled predicate changes up to the bound. Duplicated/unchanged trees are omitted; every scored trial is still charged even when another search step evaluated an identical policy. Candidate-pool truncation gives required format and stop edits priority. A final budget-limited step can score fewer trials and records the actual count.

Replay every trial on the entire SEARCH pool, with U = final accuracy - 0.01 * mean calls. The coefficient is a frozen preference, not a measured optimum. Exact utility comparisons use integer numerator 100*number_correct - total_calls, divided by 100*N. A correct individual response receives no special credit if the full-policy answer stays wrong. Choose maximum U, then lower total calls, smaller tree size, and lexicographic policy SHA256. With probability 0.20 choose uniformly from the scored legal trial pool instead, allowing exploratory and potentially harmful moves; the best trial is also allowed to be worse than the base. The exploration setting is fixed before final inference. If no distinct legal trial can be built, use a logged ordinary random mutation at the same active site with reason no_legal_distinct_trial_edits. No global error statistics are substituted for state-specific utility guidance. G3 deliberately retains its original global fallback as the legacy ablation, reported separately from random fallback.

G2 changes the proposal mechanism in two linked ways relative to G1: bounded format/stop/predicate/subtree trial construction and utility ranking, whereas G1 retains the original random proposal distribution at active locations. Thus G2 versus G1 evaluates the complete informed proposal mechanism, not utility ranking alone under an otherwise identical trial-pool distribution. G1 versus G0 primarily isolates observed/legal-location sampling. This unavoidable distribution difference is disclosed; no additional conditions are added after test inspection.

The utility hill-climbing control uses the SAME active-site rule, trial generator, complete-policy scorer, exact utility and tie rules as G2. It starts at the highest-utility shared initial candidate and accepts a trial only if its deterministic utility/cost/size/hash rank improves. After eight stagnant steps it restarts from a seeded choice among shared initial trees. It has no 20% worse-move exploration or crossover; this distinction defines the local-improvement control. All scored probes and base replays consume the same budget.

Fair accounting and resume

Every condition/seed, including hill climbing, has 4,096 full-search-policy evaluation equivalents. Each scored initial, base, trial or terminal budget-audit replay is charged one unit, including duplicates and discarded trials. Each unit replays all 512 search questions; total question-policy replays therefore equal raw scored evaluations times 512. Visitation masks and final predictions/calls are produced in the same pass, so they are not hidden uncharged replays. Labels are used only outside that label-free pass. Base replays allow comparing mutation effects after crossover and are charged consistently. G3 rescue weights reuse that scored base result; arithmetic over these stored predictions is not a new replay.

Discarded G2 probes do not enter the validation candidate archive. The archive contains shared initial

policies and selected offspring, matching the legacy archive concept. Hill climbing archives its initial pool and accepted moves. Raw scored candidate hashes, roles, trial scores and number of accepted offspring are all saved. Equal surviving-offspring counts are not described as equal compute. A single leftover budget unit is explicitly recorded as a remainder audit. Validation evaluations, baseline selection and post-search selected-policy diagnostics are measured separately and do not feed back into search. Diagnostic search replays after selection are explicitly counted in FROZEN.json; no trial scoring is hidden there.

SQLite commits each completed scored result under a deterministic step/role key, with policy hash, prediction/call arrays, base-node visit masks, utility and timing. Pending offspring plans, post-proposal RNG state, population, pending children and budget counts are durable. Resume reuses completed scores and continues the same trial plan, preserving counted records and random choices. Attempt rows expose interrupted unknown work; such overhead can exceed completed nominal units. Configuration/data/source/package drift is refused. Concurrent runs are refused with a lock. Finishing on a partial offspring generation is logged separately; standard survival is applied to the completed partial batch.

Baselines and validation selection

Retain selected singles, all 31 ascending-order fixed subsets, ten fixed triples, full-five voting, original-order agreement and the original seven greedy candidates. The legacy greedy candidates were fitted on exactly the reused search cache and are not reconstructed using final outcomes. Add every 5*4*3 = 60 ordered triples of distinct formats. Query the first two; stop only if both valid canonical answers agree; otherwise query the third and apply the SAME vote/tie/invalid rule. The original greedy is historical and has no newly matched search budget; the new utility hill control supplies the matched-budget local-search comparison. Exhaustive triples/subsets are inexpensive development-only enumerations.

Select separately for caps one, two and three using the identical prior validation rule: highest validation accuracy among candidates with mean calls <=cap; then lower mean calls, fewer nodes, smaller policy hash. If none is feasible, choose lowest calls, then highest accuracy, size and hash, and explicitly mark infeasibility. Caps are validation allowances, not requirements to spend that many calls and not exact final cost matches. Primary point is cap three. Candidate validation accuracy is a selection statistic and can overfit, not an unbiased estimate. Utility influences proposals and hill acceptance but does not replace validation accuracy/call selection.

Freeze, evaluation and analysis

One resumable entry point verifies manifest/cache compatibility, runs baselines and searches, selects on validation only, freezes all exports/configuration/source/data/package/analysis hashes, then generates the 1,000*5 final singleton responses. It evaluates only frozen exports, saves paired question predictions and produces tables, plots, mechanism diagnostics and a factual report. A completed new-final matrix never selects new policies. The old 486 questions do not enter this final evaluation. Only COMPLETE.json marks completion; incomplete records or smoke outcomes are not final results.

Primary comparisons are G1-G0, G2-G1 and G2-G3. At the primary cap, G2-G1 reports ALL of accuracy difference, mean-call difference and U difference; requested tokens, invalids, individual seeds and comparisons against selected ordered triples, old greedy and utility hill climbing are also retained. Additional caps are descriptive secondary points. Resample whole questions in 2,000 paired bootstrap replicates with seed 291029, averaging fitted-seed outcomes within each question before aggregate contrasts. Use the same question resamples across methods. Intervals condition on fitted policies, not search retraining; seed mean/SD/range and per-seed contrasts are separate. Do not treat seed-question combinations as independent or marginal interval overlap as a significance test. Achieved cost is always shown, and the final frontier is descriptive, never used to reselect policies.

Required mechanism logs include every base-policy node's visit count, zero-visit-site fraction, random/global-fallback frequency and reason, mutation fractions changing predictions and calls, every scored trial and chosen mutation's U delta, search-to-validation gaps, probes per selected offspring and actual accepted local-search moves. Selected offspring means entry into an evolutionary offspring batch, before survival, not proof of population survival. Report raw evaluation counts, full question-policy replays, unique candidate hashes, archives, full/partial generations, replay timing, CPU process time and search wall time. Cache costs distinguish historical reused search/validation calls from newly executed inference; live verification is separate. Requested-call savings in replay are not proof of deployment latency savings.

Boundaries and submission integration

Do not redesign after inspecting the new final outcomes, start another direction, claim that evolution is necessary for agreement stopping, or claim novelty for adaptive stopping, semantic/informed mutation, or utility search. A negative result is acceptable. Preserve both completed studies, label this follow-up as a response to their findings, and integrate the generated methods/results section into the final course report. The existing thematic review contains 22 primary-source recent works; preprint and formal publication status

are distinguished. Manual editorial integration of the combined report, reference/style review and final submission packaging remain writing tasks for 30 October; generated files are not an automatic course submission.

The GPU is inspected at launch and again immediately before first model loading after CPU search. No GPU memory is allocated during search, and an earlier eligibility snapshot is not treated as a reservation. The selected inference UUID is exposed before the first CUDA import/allocation. Placement requires >=20 GiB free and <=60% current utilization, prefers idle devices, and records bounded sharing when necessary. The inherited loader caps Torch allocation at 40% and rechecks >=18 GiB free. Existing processes/windows/environments are preserved.

Literature synthesis and verified primary-source references

Checked 28 October 2026 against primary author preprints, publisher records, and conference proceedings. Twenty-two relevant works below have a 2024–2026 preprint or publication; a 2023 preprint counts only when its 2024 conference publication is identified. Foundational older references are listed separately and do not count toward twenty. Verification covers bibliographic records and described mechanisms, not an independent replication of their numerical claims. No paper's headline gain is used as an expected outcome for this project.

Evolution and feedback: what should the operator search?

Evolutionary LLM research operates on several representations. EvoPrompt [1] and Promptbreeder [2] evolve instruction text; the latter also evolves mutation prompts. OPRO [10] uses scored histories to propose improvements, while EvoLLM [3] investigates in-context optimization operators themselves. EoH [4] evolves heuristic ideas and executable programs, Eureka [6] evolves reward code, and ADAS [7] searches agent programs. Model-merging evolution [8] changes model composition rather than query behavior. The survey [9] situates both EA-enhanced LLMs and LLM-enhanced EAs. Together these establish that an evolutionary wrapper alone is insufficient as a contribution: the useful question is what information the operator exploits and whether that improves search under controlled budgets.

GEPA [5] is a particularly relevant warning against broad novelty claims: it uses feedback from trajectories and combines complementary lessons across candidate lineages. TextGrad [13] likewise exploits richer feedback than a scalar score. DSPy [11] and MIPRO [12] optimize compound LM programs, exposing credit assignment and validation-selection questions. Their representations and feedback costs are much broader than this project's compact trees. Our narrow candidate contribution is a rescue-count distribution for choosing a presentation at a mutation site, compared with uniform random mutation using identical executable representation and evaluated-candidate budgets. It uses search labels only and does not ask an LLM to invent a prompt. It should be described as an ablation of a practical integration of existing ideas, not a new general optimizer or a GEPA reproduction.

Strength: targeted feedback could reduce unproductive format proposals. Limitation: rescue of a wrong terminal prediction does not guarantee that adding the response changes a majority vote, and correlated mistakes may persist. State-specific counts can be noisy, so the fallback is declared rather than tuned using validation outcomes. Greedy construction receives the same search statistics, testing whether evolution adds anything beyond local construction.

Presentation sensitivity, response contracts, and ensemble diversity

Sclar et al. [14] show that apparently superficial formatting choices can affect model performance and introduce FormatSpread. Wang et al. [15] distinguish generated text answers from first-token likelihood evaluation for multiple-choice robustness. These findings motivate a small fixed pool, exact preservation of option text, canonical identity remapping, and an explicit parser. This experiment evaluates generated letters; it does not infer that a first-token evaluation would behave identically.

MoA [18] aggregates model outputs in layers, whereas Self-MoA [19] cautions that adding different models can sacrifice source quality even while increasing diversity. Soft self-consistency [20] considers score-based selection when exact answer matches are sparse. Their common lesson is that raw diversity is not equivalent to useful accuracy gain. The exploratory pilot's oracle coverage indicates only the existence of correct answers in the pool; it is not implementable without labels. Fixed-subset enumeration and single-format selection are therefore essential controls. Our four-option canonical answers make plurality voting natural, without extra LLM aggregation calls or probability features. No claim is made that format diversity is equivalent to model diversity.

Strength: equivalent presentations isolate a reproducible source of disagreement within one frozen model. Limits: a finite pool can miss other useful perturbations, rotations may alter behavior despite preserving intended semantics, and all simple ensembles can fail together. Oracle coverage is only an upper bound and

cannot be a deployment comparator.

Adaptive computation and cost: why stopping is already established

RouteLLM [16] learns model routing from preference data. RouterBench [17] makes cached inference outcomes useful for systematic routing evaluation. Snell et al. [21] emphasize that effective test-time computation depends on problem difficulty. ReASC [22] extends count-based adaptive sampling using response reliability. The older Adaptive-Consistency and FrugalGPT papers (below) directly establish agreement stopping and learned cascades before this project. Consequently neither adaptive allocation nor cheap early exit is novel here.

This project restricts observations to queried canonical predictions, vote counts, invalid indicators, queried formats and remaining calls. It uses no learned confidence estimator, hidden-state features, reference answer at inference, or access to unqueried predictions. This makes policy behavior auditable but can limit headroom relative to richer routers. Equal call caps do not imply equal tokens or latency: formatting changes input length, batching changes execution, and generated responses can vary with execution context. We separately report matrix-generation work, CPU search, requested deployment tokens/calls, and a small measured live sample. A cached matrix can support reproducible policy comparisons but cannot establish a production latency improvement.

Research gap and falsifiable assessment

A modest, testable opportunity is whether search-only error complementarity helps an evolutionary operator discover compact presentation policies that retain accuracy with fewer requested calls. The controlled comparison is guided versus uniform mutation over five search seeds at predeclared validation cost caps. The stronger practical question is whether either beats greedy and validation-selected fixed subsets at the achieved final costs. Failure on either comparison is retained. Existing work makes publication-level novelty uncertain; this review motivates a course-project contribution and explicitly does not establish a first-in-literature claim.

Recent references: 22 works

ID	Reference and verified year/status	Primary source	Role and distinction
1	Guo et al. EvoPrompt: Connecting LLMs with Evolutionary Algorithms Yields Powerful Prompt Optimizers. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2309.08532), proceedings (https://proceedings.iclr.cc/paper_files/paper/2024/hash/9156b0f6dfa9bbd18c79cc459ef5d61c-Abstract-Conference.html)	Prior prompt evolution; here the instruction is fixed.
2	Fernando et al. Promptbreeder: Self-Referential Self-Improvement via Prompt Evolution. ICML 2024.	PMLR (https://proceedings.mlr.press/v235/fernando24a.html)	Prior adaptation of mutation prompts; no claim of inventing adaptive mutation.
3	Lange, Tian, Tang. Large Language Models As Evolution Strategies. Preprint 2024.	arXiv (https://arxiv.org/abs/2402.18381)	LLMs as recombination operators; ours is conventional CPU tree search.
4	Liu et al. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. 2024.	Author preprint (https://arxiv.org/abs/2401.02051)	Prior search over executable heuristics; no heuristic-design novelty claimed.
5	Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. Preprint 2025, v2 February 2026.	arXiv (https://arxiv.org/abs/2507.19457v2)	Reflective feedback and complementary candidate lessons are close conceptual precedents.
6	Ma et al. Eureka: Human-Level Reward Design via Coding Large Language Models. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2310.12931)	Evolutionary reward-code generation; distinct task and representation.
7	Hu, Lu, Clune. Automated Design of Agentic Systems. ICLR 2025; preprint 2024.	Author preprint (https://arxiv.org/abs/2408.08435)	Search over agents; our inference features and actions are deliberately bounded.
8	Akiba et al. Evolutionary optimization of model merging recipes. Nature Machine Intelligence 7, 195 –204, 2025.	Publisher (https://www.nature.com/articles/s42256-024-00975-8)	Evolutionary model composition; ours changes presentation queries with frozen weights.
9	Wu et al. Evolutionary Computation in the Era of Large Language Model: Survey and Roadmap. Preprint 2024; IEEE TEC 2025 (course brief citation).	Author preprint (https://arxiv.org/abs/2401.10034)	Frames both directions of the EC–LLM intersection.
10	Yang et al. Large Language Models as Optimizers. ICLR 2024; preprint 2023, revised April 2024.	Author preprint (https://arxiv.org/abs/2309.03409)	Score-history-driven optimization; ours uses structured search-only error statistics.
11	Khattab et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. ICLR 2024; preprint 2023.	Author preprint (https://arxiv.org/abs/2310.03714)	Compound program optimization; no new claim for program-level search.
12	Opsahl-Ong et al. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. EMNLP 2024.	ACL proceedings (https://aclanthology.org/2024.emnlp-main.525/)	MIPRO credit assignment and surrogate optimization; not reproduced.

13	Yuksekgonul et al. TextGrad: Automatic Differentiation via Text. Preprint 2024.	Author preprint (https://arxiv.org/abs/2406.07496)	Rich textual optimization feedback; our operator uses no extra inference feedback.
14	Sclar, Choi, Tsvetkov, Suhr. Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. ICLR 2024; preprint revised July 2024.	Author preprint (https://arxiv.org/abs/2310.11324)	Format sensitivity/FormatSpread; formatting effects already established.
15	Wang et al. Look at the Text: Instruction-Tuned Language Models are More Robust Multiple Choice Selectors than You Think. COLM 2024.	Author preprint (https://arxiv.org/abs/2404.08382)	Generated-text versus first-token evaluation motivates response-contract limits.
16	Ong et al. RouteLLM: Learning to Route LLMs with Preference Data. Preprint 2024, revised 2025.	arXiv (https://arxiv.org/abs/2406.18665)	Cost-quality routing prior; uses preferences and model routing, not format trees.
17	Hu et al. RouterBench: A Benchmark for Multi-LLM Routing System. Preprint 2024.	arXiv (https://arxiv.org/abs/2403.12031)	Cached inference outcomes for routing evaluation; supports replay methodology but not live latency claims.
18	Wang et al. Mixture-of-Agents Enhances Large Language Model Capabilities. Preprint 2024.	arXiv (https://arxiv.org/abs/2406.04692)	Layered multi-model aggregation; our voting requires no aggregation-model call.
19	Li, Lin, Xia, Jin. Rethinking Mixture-of-Agents: Is Mixing Different Large Language Models Beneficial? Preprint 2025.	arXiv (https://arxiv.org/abs/2502.00674)	Self-MoA studies diversity-quality trade-off; useful warning against assuming complementarity helps.
20	Wang, Prasad, Stengel-Eskin, Bansal. Soft Self-Consistency Improves Language Model Agents. ACL 2024.	ACL proceedings (https://aclanthology.org/2024.acl-short.28/)	Soft scoring for varied valid outputs; ours has four fixed canonical identities. ACL web title says “Models Agents”; PDF/preprint use “Model Agents.”
21	Snell, Lee, Xu, Kumar. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. Preprint 2024.	arXiv (https://arxiv.org/abs/2408.03314)	Difficulty-dependent allocation; no claim of inventing adaptive test-time compute.
22	Kim, Yang, Min, Jung. Reliability-Aware Adaptive Self-Consistency for Efficient Sampling in LLM Reasoning. 2026, v2 April, ACL 2026 per author record.	arXiv (https://arxiv.org/abs/2601.02970)	Reliability-based stopping; omitted confidence features limit comparability.

Older foundations and assets, excluded from recent-paper count

- Aggarwal, Madaan, Yang, Mausam. Let's Sample Step by Step: Adaptive-Consistency for Efficient Reasoning and Coding with LLMs. EMNLP 2023. Primary paper (https://arxiv.org/abs/2305.11860). Prior adaptive agreement sampling; the fixed rule here is a transparent simple control, not a full reproduction.
- Chen, Zaharia, Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. 2023. Primary preprint (https://arxiv.org/abs/2305.05176). Learned cost-quality cascades precede this work.
- Zhou, Wu, Tang. Ensembling neural networks: Many could be better than all. Artificial Intelligence 137, 239–263, 2002. Publisher (https://www.sciencedirect.com/science/article/pii/S000437020200190X), author code (https://www.lamda.nju.edu.cn/code_GASEN.ashx). Genetic selective ensembles are long established.
- Deb, Pratap, Agarwal, Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE TEC 6, 182–197, 2002. Author publication record (https://www.egr.msu.edu/~kdeb/reports.shtml), DOI 10.1109/4235.996017. We use non-dominated sorting and crowding but a simple front-construction implementation and rank/hash tournament; call it NSGA-II-style, not an exact implementation.
- Pezeshkpour, Hruschka. Large Language Models Sensitivity to the Order of Options in Multiple-Choice Questions. 2023. Primary preprint (https://arxiv.org/abs/2308.11483). Option rotation sensitivity is prior work.
- Mihaylov, Clark, Khot, Sabharwal. Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering. EMNLP 2018. Paper (https://arxiv.org/abs/1809.02789), official code/data (https://github.com/allenai/OpenBookQA). This project uses questions and choices without supplying the annotated supporting fact, so it evaluates closed-book answering on OpenBookQA.
- Qwen2.5-3B-Instruct official model card (https://huggingface.co/Qwen/Qwen2.5-3B-Instruct). Model/tokenizer revision aa8e72537993ba99e69dfaafa59ed015b17504d1; fixed instruction and weights.

TARE is excluded because the previous project documented conflicting withdrawal/status records. No empirical TARE claim supports this project.